Assignment Title

Barber Booking System

Group Members:

Rodrigo Xavier Ramos (20059753)
Fady Abdelrahman Mohamed (20053593)

Lecturer:
Dr. Basel Magableh

Module/Subject Title
WEB AND CLOUD APPLICATION DEVELOPMENT (B8IT146)

Submission Date: 12 April 2026
Academic Year 2025/2026

Acknowledgment

We would like to acknowledge the collaborative effort that made this assignment possible.

Working together as a team has been a rewarding experience that taught us the value of effective communication, mutual support, and shared responsibility. By combining our individual strengths and keeping open dialogue throughout the process, we were able to tackle challenges and produce work that reflects our collective dedication.

Each team member brought unique perspectives and skills to the table. Through helpful discussions and regular feedback, we complemented each other's abilities and improved the quality of our work. This collaborative approach not only enhanced our final output but also deepened our understanding of information systems development and management.

We are grateful for the commitment and professionalism we showed toward one another during the research, analysis, and documentation phases.

Finally, as **Phil Jackson** once said:

"The strength of the team is each individual member. The strength of each member is the team."

Declaration by Students

We, Rodrigo Xavier Ramos and Fady Abdelrahman Mohamed, hereby declare that the work presented herein is original work done by us and has not been previously submitted for academic assessment at this or any other institution. Any literature, data, or work done by others and cited within this assignment has been given due to acknowledgement and listed in the bibliography section.

Contents

Introduction 5
Purpose 5
Key Features 5
System Architecture 6
Architecture Pattern 6
Component Overview 6
Technology Stack 6
Frontend Technologies 6
Backend Technologies 7
Database 7
Database Design 8
Entity Relationship Diagram (ERD) 8
Table Schemas 8
Users Table 8
Barbers Table 9
Services Table 9
Appointments Table 10
API Endpoints 10
Authentication Endpoints 10
POST /api/auth/register 10
POST /api/auth/login 11
Frontend Components 14
Installation Guide 17
Prerequisites 17
Backend Setup 17
Frontend Setup 18
Team Contributions 18
Conclusion 20
References 20

Introduction

Our project, the Barber Booking System, was born out of a simple need we identified during the "Web and Cloud Application Development" module (B8IT146) at Dublin Business School: the need to make booking a haircut as easy as ordering a coffee. We wanted to build a modern, full-stack solution that replaces the old-fashioned phone calls and walk-ins with a smooth, digital experience where customers can manage everything from their phones or laptops.

To make this work, we didn't just want a simple website; we aimed for a professional-grade architecture. We built the frontend using React 18 and Vite to ensure the site is fast and responsive, styled with Bootstrap 5. On the backend, we developed a robust Node.js and Express server that handles over fifteen different API endpoints. Security was a huge priority for us, so we implemented JWT for logins and bcrypt to make sure all user passwords are encrypted and safe. All the data from the barbers' schedules to the specific services offered is stored and organised in a MySQL (migrated to SQL Server) database that we carefully designed to be stable and scalable.

In the following pages, we'll walk you through exactly how we built this system from the initial database sketches to the final API testin complete with diagrams and screenshots of the finished product.

Purpose

The Barber Booking System is a full-stack web application designed to streamline the appointment booking process for barbershops. It allows customers to:

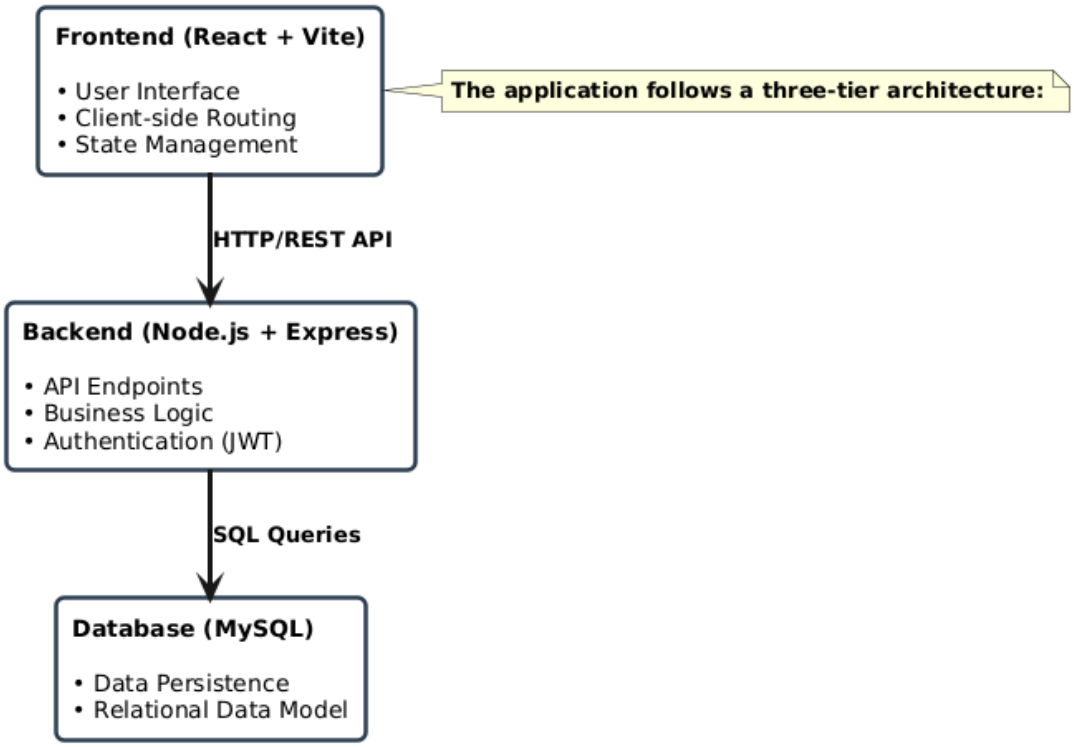
- Browse available services and barbers
- Book appointments online
- Manage their bookings
- View appointment history

Key Features

- **User Authentication**: Secure login and registration with JWT
- **Service Catalog**: Browse all available barber services
- **Barber Profiles**: View detailed information about each barber
- **Appointment Booking**: Easy-to-use booking system
- **Appointment Management**: View, track, and cancel appointments
- **Dashboard**: Overview of user's booking statistics
- **Responsive Design**: Works seamlessly on desktop, tablet, and mobile

System Architecture

Architecture Pattern



Component Overview

Frontend: Single Page Application (SPA) built with React
Backend: RESTful API server with Express.js
Database: MySQL relational database
Authentication: JWT-based token authentication
Password Security: bcrypt hashing

Technology Stack

Frontend Technologies

Technology	Purpose
React	UI Framework
React Router DOM	Client-side Routing
Vite	Build Tool
Bootstrap 5	UI Components & Styling
React Bootstrap	React Bootstrap Components

Axios	HTTP Client

Backend Technologies

Technology	Purpose
Node.js	Runtime Environment
Express.js	Web Framework
MySQL2	Database Driver
bcryptjs	Password Hashing
jsonwebtoken	JWT Authentication
express-validator	Input Validation
dotenv	Environment Variables
cors	Cross-Origin Resource Sharing

Database

Technology	Purpose
MySQL	Relational Database Management System

Database Design

Entity Relationship Diagram (ERD)

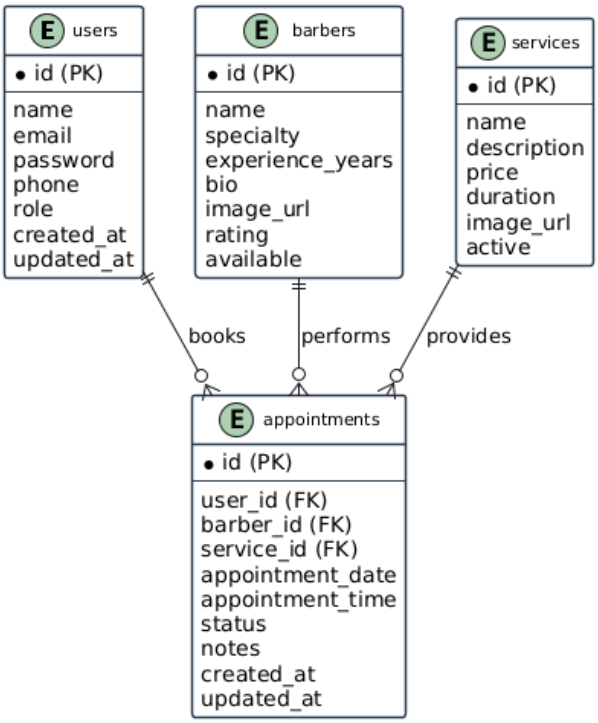


Table Schemas

Users Table

```sql
-- Table 1: Users
IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'users')
BEGIN
    CREATE TABLE users (
        id INT IDENTITY(1,1) PRIMARY KEY,
        username NVARCHAR(50) UNIQUE NOT NULL,
        name NVARCHAR(100) NOT NULL,
        email NVARCHAR(100) UNIQUE NOT NULL,
        password NVARCHAR(255) NOT NULL,
        phone NVARCHAR(20),
        role NVARCHAR(20) DEFAULT 'customer' CHECK (role IN ('customer', 'admin')),
        created_at DATETIME2 DEFAULT GETDATE(),
        updated_at DATETIME2 DEFAULT GETDATE()
    );

    CREATE INDEX idx_email ON users(email);
    CREATE INDEX idx_username ON users(username);
    CREATE INDEX idx_role ON users(role);
END
GO
```

Barbers Table

```sql
-- Table 2: Barbers
IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'barbers')
BEGIN
    CREATE TABLE barbers (
        id INT IDENTITY(1,1) PRIMARY KEY,
        name NVARCHAR(100) NOT NULL,
        specialty NVARCHAR(100),
        experience_years INT DEFAULT 0,
        bio NVARCHAR(MAX),
        image_url NVARCHAR(255),
        rating DECIMAL(3,2) DEFAULT 0.00,
        available BIT DEFAULT 1,
        created_at DATETIME2 DEFAULT GETDATE(),
        updated_at DATETIME2 DEFAULT GETDATE()
    );

    CREATE INDEX idx_available ON barbers(available);
    CREATE INDEX idx_rating ON barbers(rating);
END
GO
```

Services Table

```sql
-- Table 3: Services
IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'services')
BEGIN
    CREATE TABLE services (
        id INT IDENTITY(1,1) PRIMARY KEY,
        name NVARCHAR(100) NOT NULL,
        description NVARCHAR(MAX),
        price DECIMAL(10,2) NOT NULL,
        duration INT NOT NULL,
        image_url NVARCHAR(255),
        active BIT DEFAULT 1,
        created_at DATETIME2 DEFAULT GETDATE(),
        updated_at DATETIME2 DEFAULT GETDATE()
    );

    CREATE INDEX idx_active ON services(active);
    CREATE INDEX idx_price ON services(price);
END
GO
```

Appointments Table

```sql
-- Table 4: Appointments
IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'appointments')
BEGIN
    CREATE TABLE appointments (
        id INT IDENTITY(1,1) PRIMARY KEY,
        user_id INT NOT NULL,
        barber_id INT NOT NULL,
        service_id INT NOT NULL,
        appointment_date DATE NOT NULL,
        appointment_time TIME NOT NULL,
        status NVARCHAR(20) DEFAULT 'pending' CHECK (status IN ('pending', 'confirmed', 'completed', 'cancelled')),
        notes NVARCHAR(MAX),
        created_at DATETIME2 DEFAULT GETDATE(),
        updated_at DATETIME2 DEFAULT GETDATE(),
        FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
        FOREIGN KEY (barber_id) REFERENCES barbers(id) ON DELETE CASCADE,
        FOREIGN KEY (service_id) REFERENCES services(id) ON DELETE CASCADE
    );
```

API Endpoints

Authentication Endpoints

POST /api/auth/register

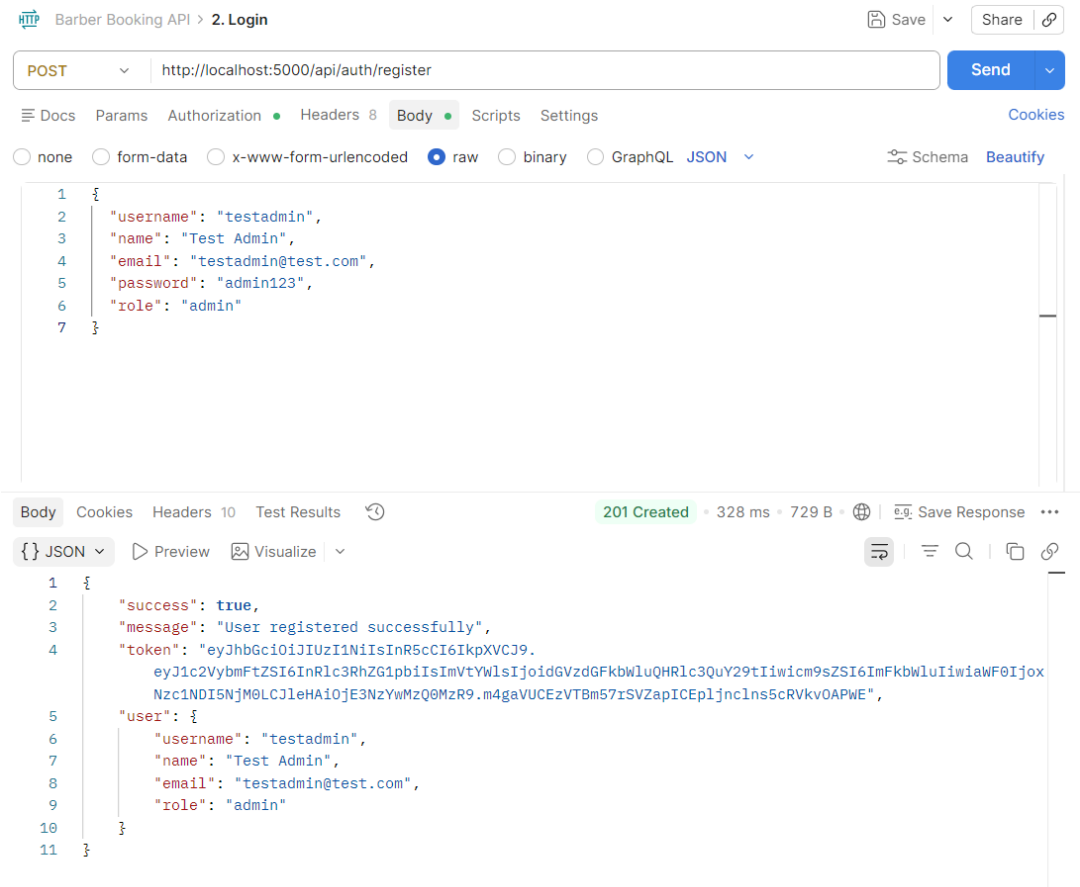


Figure 1 Testing User Registration Endpoint - POST /api/auth/register

POST /api/auth/login

Barber Booking API > 2. Login

Save | Share

POST http://localhost:5000/api/auth/login Send

Docs | Params | Authorization | Headers 8 | Body | Scripts | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Schema | Beautify

```
1  {
2    "username": "testadmin",
3    "password": "admin123"
4  }
```

Body | Cookies | Headers 10 | Test Results | 200 OK | 260 ms | 729 B | Save Response

{} JSON | Preview | Visualize


```
1  {
2      "success": true,
3      "message": "Login successful",
4      "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
           eyJpZCI6NCwidXNlcm5hbWUiOiJ0ZXN0YWRtaW4iLCJlbWFpbCI6InRlc3RhZG1pbkB0ZXN0LmNvbSIsInJvbGUiOiJhZG1pbiIs
           ImlhdCI6MTc3NTQzMDA0NSwiZXhwIjoxNzc2MDM0ODQ1fQ.fhs6QGXQwPDCYDs9ZgvWd8gT72Wj8P6Qm_fn3-y1C20",
5      "user": {
6          "id": 4,
7          "username": "testadmin",
8          "name": "Test Admin",
9          "email": "testadmin@test.com",
10         "role": "admin"
11     }
12 }
```


Figure 2 Testing User Login Endpoint - POST /api/auth/login

localhost:5000/api/health

Pretty print


```
{"success":true,"message":"Barber Booking API is running","timestamp":"2026-04-05T01:49:18.446Z"}
```


Figure 3 Testing Health Check Endpoint - GET /api/health

Figure 4 Testing Get All Barbers Endpoint - GET /api/barbers

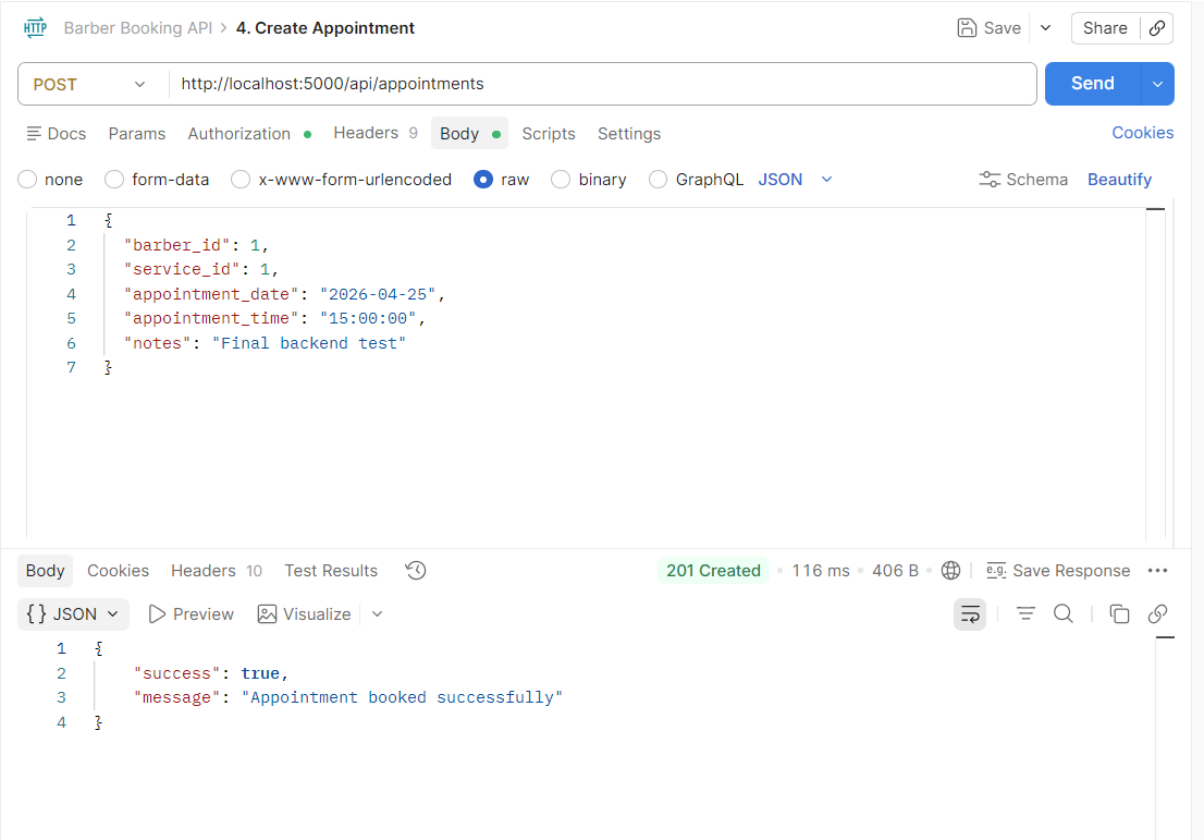


Figure 5 Testing Create Appointment Endpoint - POST /api/appointments

Barber Booking API > **5. Get My Appointments**

Save | Share

GET http://localhost:5000/api/appointments Send

Docs | Params | Authorization | Headers 7 | Body | Scripts | Settings | Cookies

Auth Type: Bearer Token

Token: ••••••••••••••••••••••••••••••••

The authorization header will be automatically generated when you send the request. Learn more about Bearer Token authorization.

Body | Cookies | Headers 10 | Test Results | 200 OK • 126 ms • 639 B | Save Response

{} JSON | Preview | Visualize


```
1  {
2      "success": true,
3      "count": 1,
4      "data": [
5          {
6              "id": 3,
7              "appointment_date": "2026-04-25T00:00:00.000Z",
8              "appointment_time": "1970-01-01T15:00:00.000Z",
9              "status": "pending",
10             "notes": "Final backend test",
11             "barber_name": "John Smith",
12             "specialty": "Classic Cuts",
13             "service_name": "Classic Haircut",
14             "price": 25,
```


Figure 6 Testing Get My Appointments Endpoint - GET /api/appointments (with Bearer Token)

Frontend Components

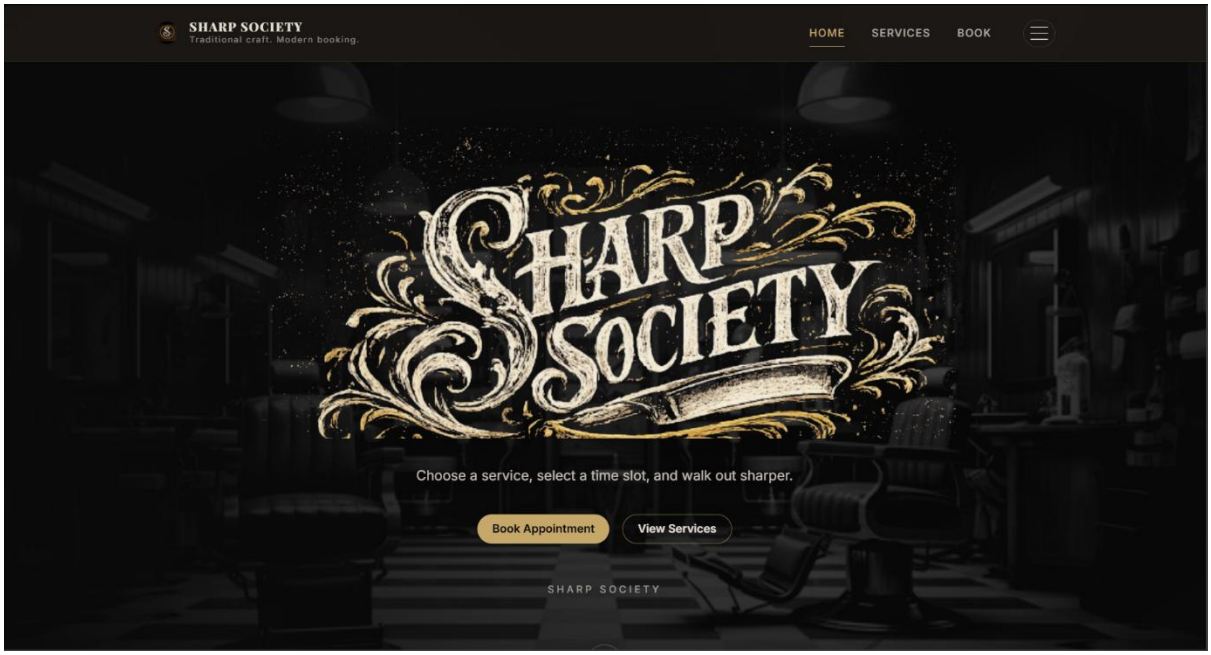


Figure 7 Homepage of Sharp Society - Barber Booking System

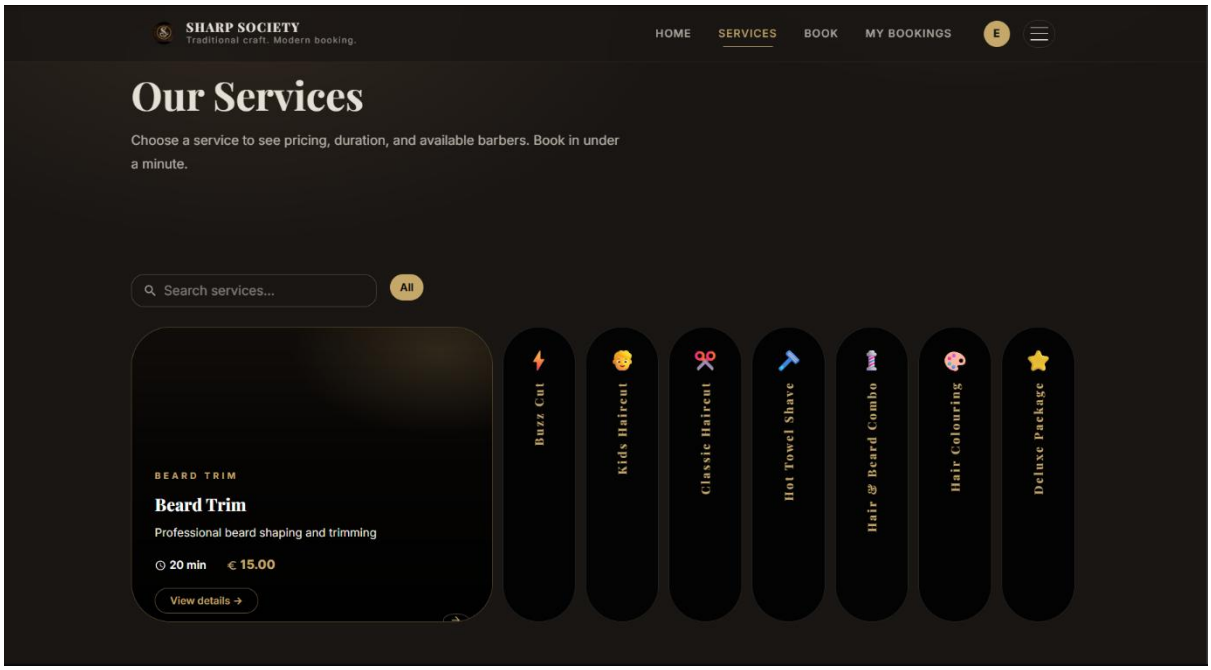


Figure 8 Services Listing Page — Sharp Society Barber Booking System

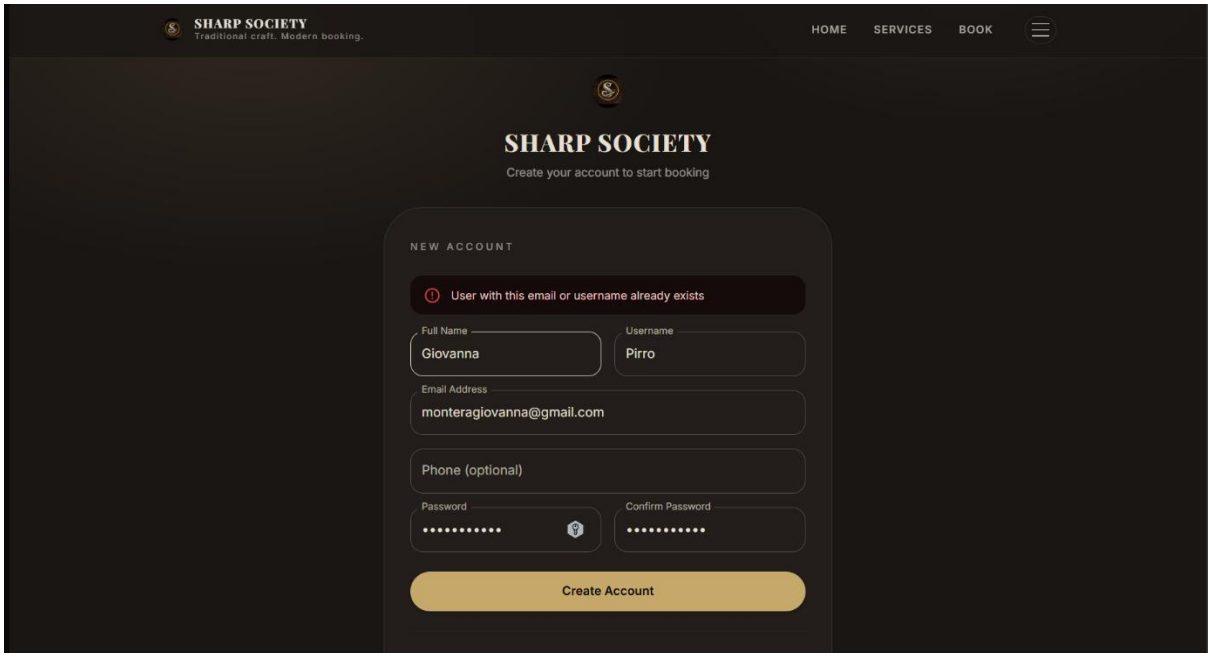


Figure 9 Registration Page with Error Handling

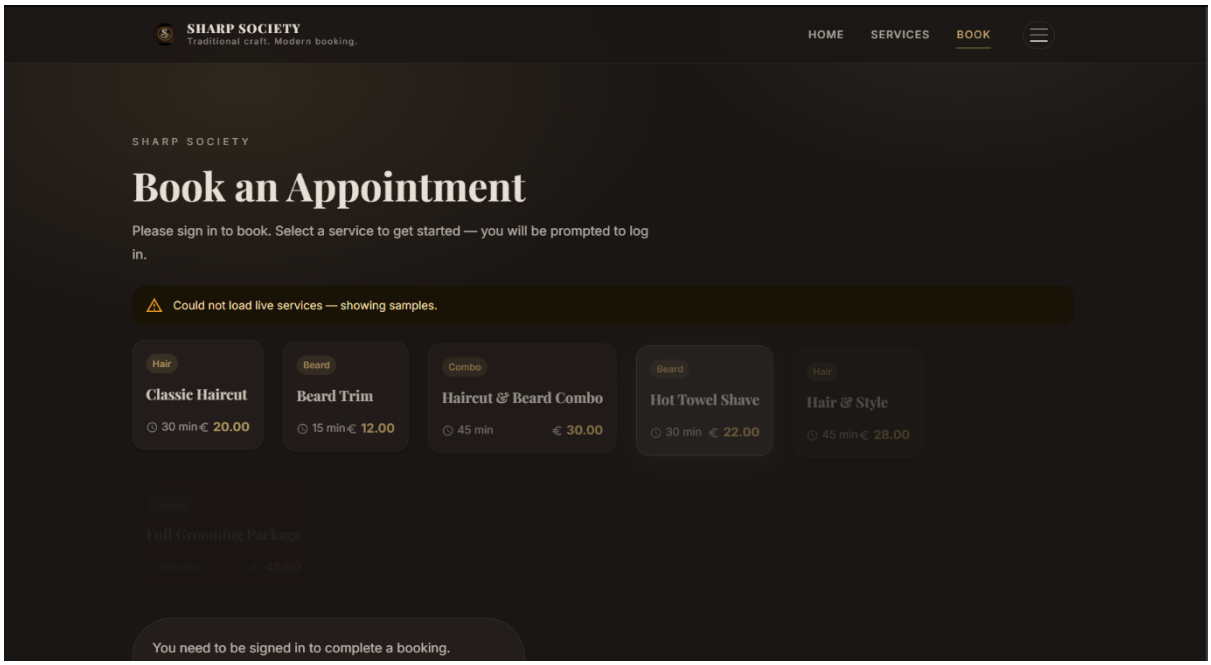


Figure 10 Book an Appointment Page Service Selection with Warning Messages

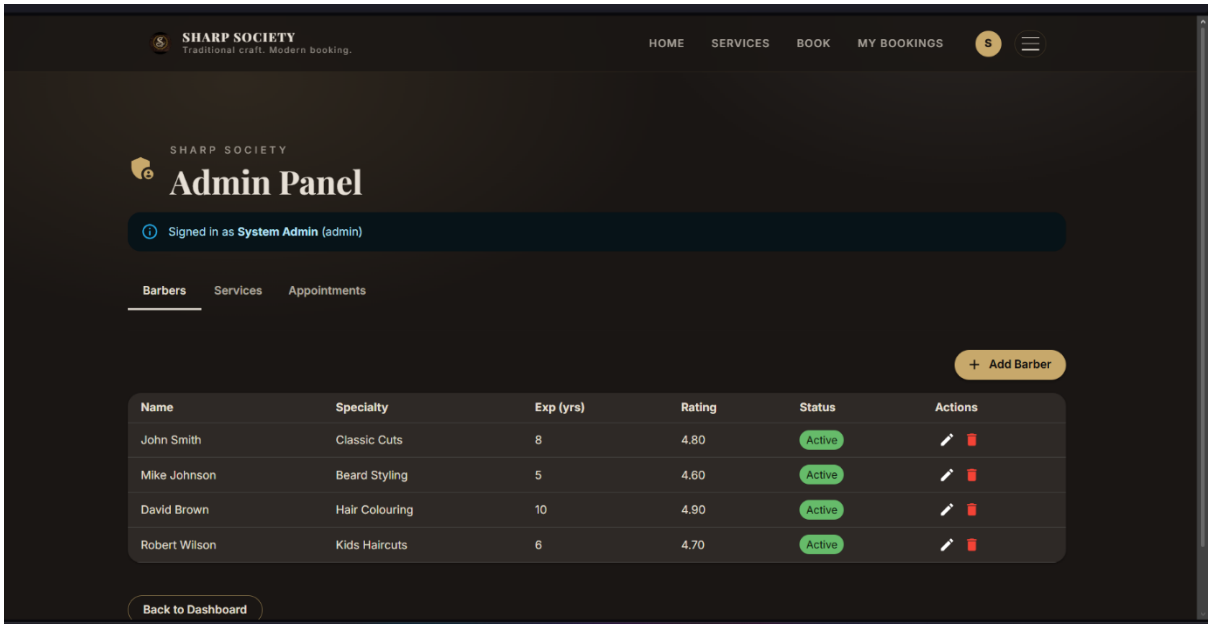


Figure 11 Admin Panel - Barbers Management Section

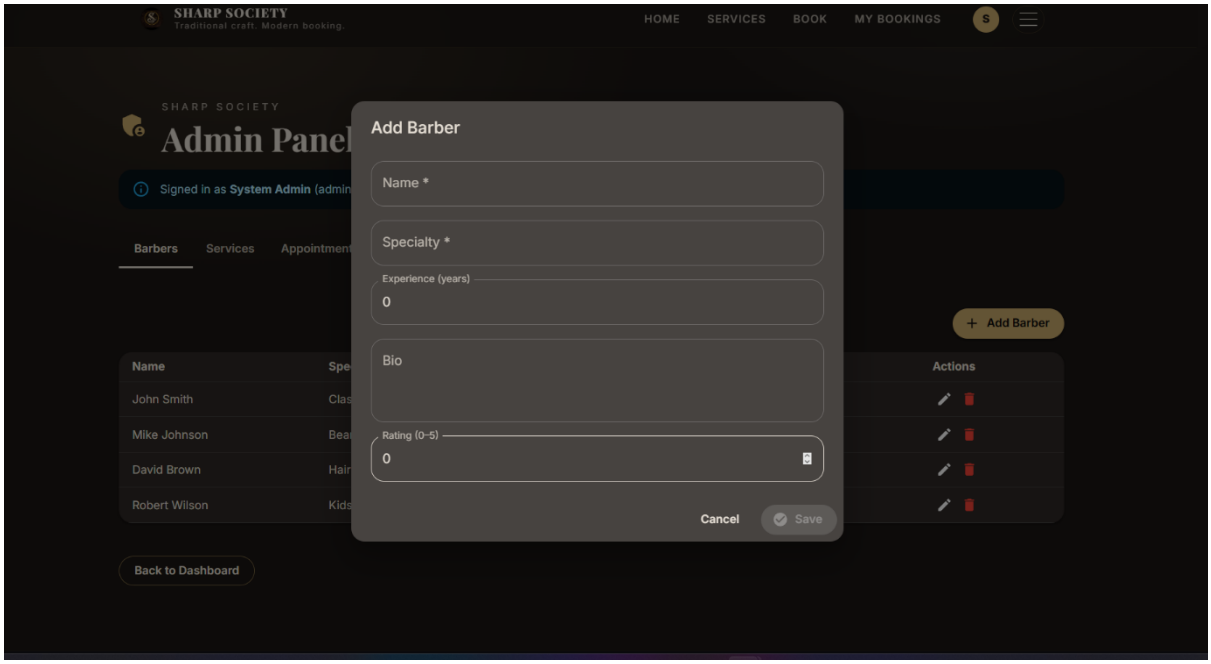


Figure 12 Admin Panel Add Barber Modal Form

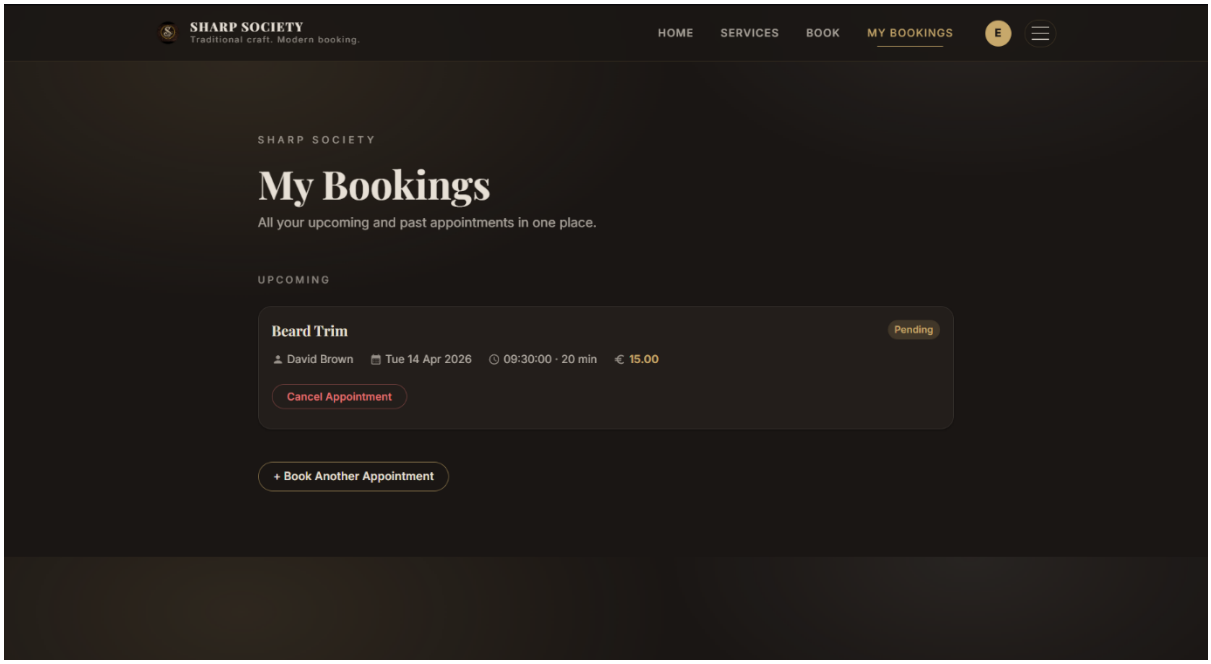


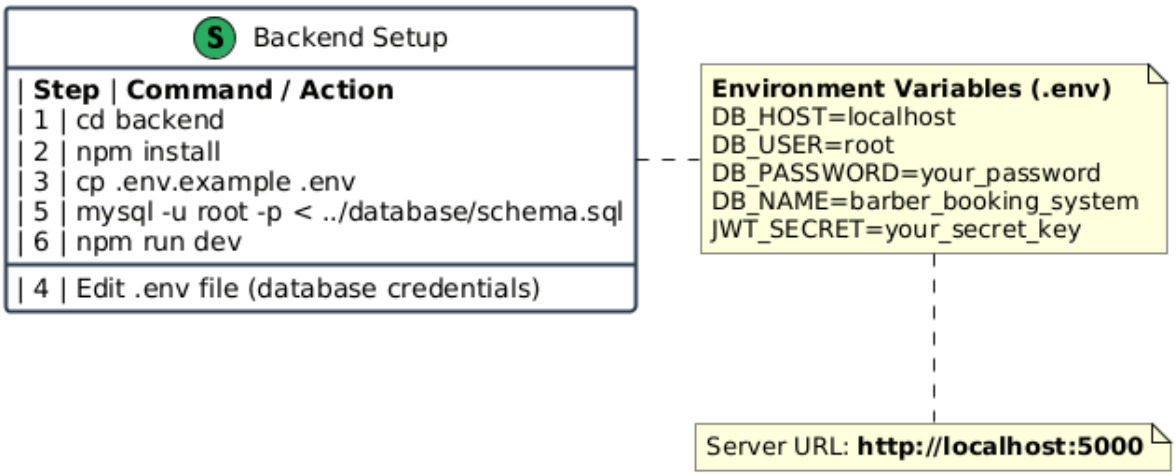
Figure 13 y Bookings Page Upcoming Appointments View

Installation Guide

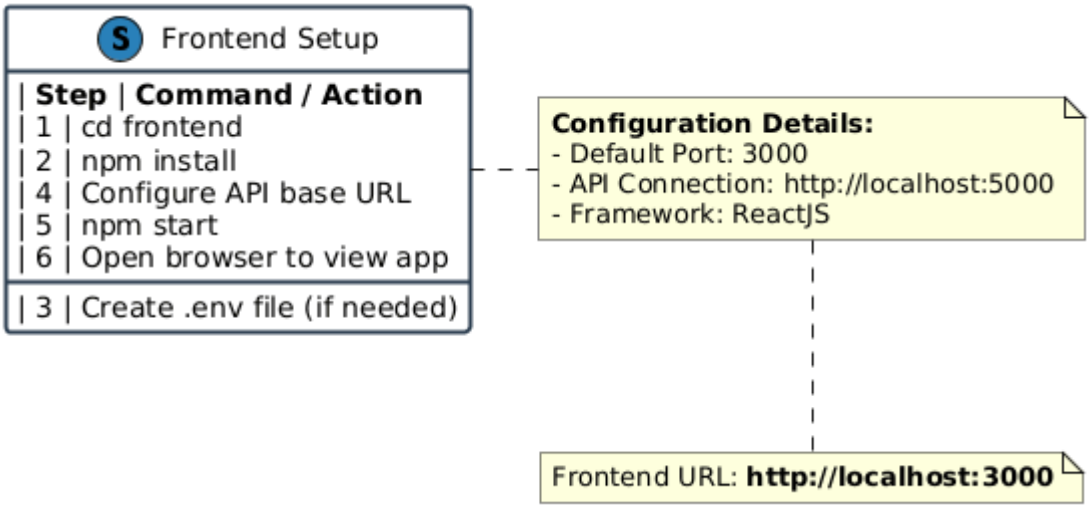
Prerequisites

- Node.js 18+ installed
- MySQL 8.0+ installed
- npm or yarn package manager

Backend Setup



Frontend Setup



Team Contributions

Member	Student ID	Role	Responsibilities	Contribution %
Fady Abdelrahman Mohamed	**20053593**	**Backend Lead & Database Architect**	• Database Design: Designed and implemented all 4 MySQL tables with proper relationships and constraints	**50%**
			• API Development: Built 15+ RESTful API endpoints using Express.js	
			• Authentication: Implemented JWT-based login/register system with bcrypt password hashing	
			• CRUD Operations: Developed full CRUD operations for Users, Barbers, Services, and Appointments	
			• Middleware: Created authentication middleware and input validation with express-validator	
			• API Testing: Tested and documented all endpoints using Postman	

Rodrigo Xavier Ramos	**20059753**	**Frontend Lead & Integration Developer**	• React Setup: Initialized the React + Vite project and configured routing, AuthContext, and Axios	**50%**
			• Shared Components: Built Navbar, Footer, and LoadingSpinner reusable components	
			• Pages 1–4: Developed Home, Login, Register, and Services List pages with full validation	
			• Pages 5–8: Developed Barbers List, Book Appointment, My Appointments, and Dashboard pages	
			• API Integration: Connected all frontend pages to the backend REST APIs	
			• Booking Feature: Implemented the full appointment booking flow with time slot selection	
			• Appointment Management: Built My Appointments page with cancel functionality and status badges	
			• Responsive Design: Ensured full responsiveness across desktop, tablet, and mobile screens	
Both Members	—	**Shared Work**	• Final Testing: End-to-end testing of all pages and features to ensure full system functionality	**Shared 50/50**
			• Report Writing: Co-authored the project documentation and technical report (split equally 50/50)	
			• Oral Defence Prep: Joint preparation of presentation and oral defence materials	

Conclusion

Building this system has been a powerful learning experience for us. It taught us firsthand how to bridge the different layers of a Full-stack application and make them work together seamlessly. We are proud of the result; the system is more than just a class project to us it's a solid, scalable foundation that we could easily expand in the future with features like online payments.

The modular architecture allows for easy maintenance and future enhancements. The clear separation of concerns between frontend, backend, and database makes it scalable and maintainable.

References

[1] F. Abdelrahman and R. Xavier Ramos, "Barber Booking System," GitHub Repository, 2026. [Online]. Available: https://github.com/FadyAbdelrahman/Barber-Booking-System.

[2] PlantUML, "PlantUML Online Editor," editor.plantuml.com, 2024. [Online]. Available: https://editor.plantuml.com/.

[3] Meta Open Source, "React — A JavaScript Library for Building User Interfaces," React Documentation, 2024. [Online]. Available: https://react.dev/.

[4] OpenJS Foundation, "Node.js Documentation," nodejs.org, 2024. [Online]. Available: https://nodejs.org/en/docs.

[5] StrongLoop Inc., "Express.js — Fast, Unopinionated, Minimalist Web Framework for Node.js," expressjs.com, 2024. [Online]. Available: https://expressjs.com/.

[6] Oracle Corporation, "MySQL 8.0 Reference Manual," dev.mysql.com, 2024. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/.

[7] Auth0 by Okta, "Introduction to JSON Web Tokens," jwt.io, 2024. [Online]. Available: https://jwt.io/introduction.

[8] The Bootstrap Team, "Bootstrap 5 Documentation," getbootstrap.com, 2024. [Online]. Available: https://getbootstrap.com/docs/5.3/.

[9] Postman Inc., "Postman API Platform Documentation," learning.postman.com, 2024. [Online]. Available: https://learning.postman.com/docs.

[10] Vitejs Team, "Vite — Next Generation Frontend Tooling," vitejs.dev, 2024. [Online]. Available: https://vitejs.dev/.